Quantitative Trading Architecture

SGLD Uncertainty Model for Reversal Zone Detection

Technical Specification: Model Structure, Training Pipeline and Mathematical Formulation

1. Overview

This document provides a rigorous technical description of the SGLD Uncertainty-Aware sequential Gated Recurrent Unit (GRU) model designed for the detection of market reversal zones (local extrema). The system targets three-class classification of market regimes: Trough (Buy Zone, class 0), Peak (Sell Zone, class 1), and Trend / Neutral (class 2).

The architecture integrates classical technical analysis feature engineering with a Bayesian approximate inference procedure based on Stochastic Gradient Langevin Dynamics (SGLD). Posterior samples collected after a deterministic burn-in phase enable Monte Carlo estimation of predictive distributions. The resulting uncertainty measures (probability spreads) are employed as risk filters within a subsequent trading execution layer.

The model was developed and evaluated on the Hang Seng Index (^HSI) over the period 2011–2021, using chronological train / validation / test splits (504 trading days each for validation and test).

2. System Architecture & Training Pipeline

Figure 1 presents the end-to-end data and control flow of the system, following the conventional structure used in machine-learning research papers. The diagram separates the offline training phase (yellow region) from the online inference and decision stages.

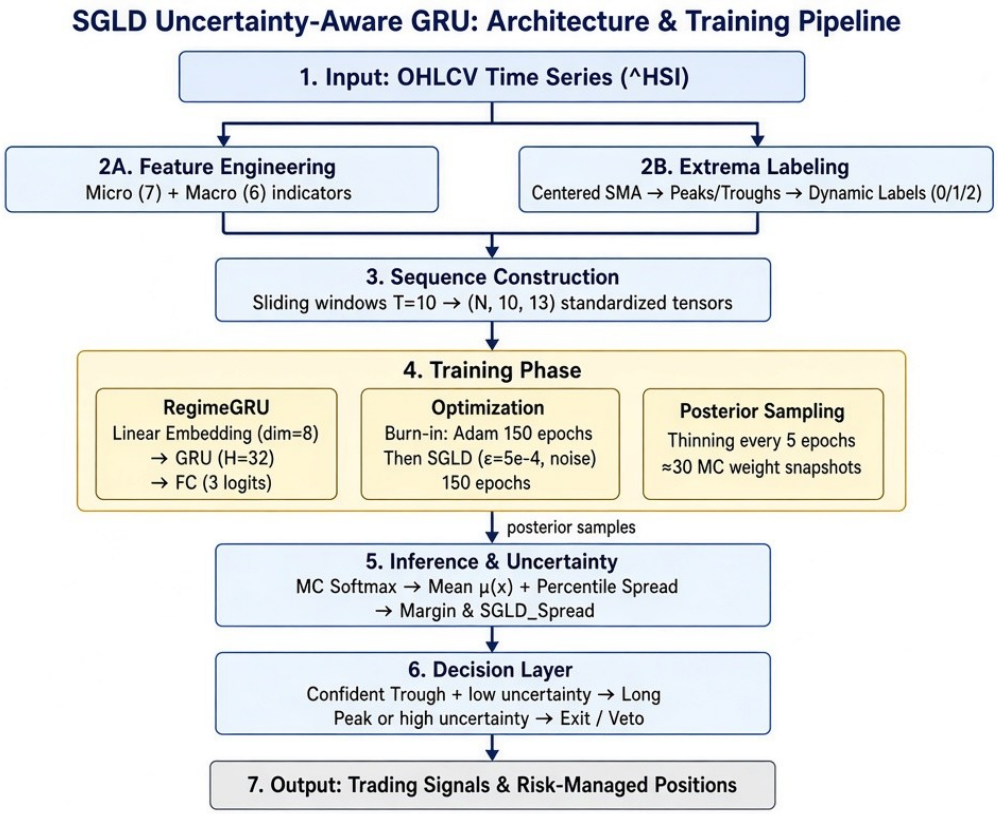

Figure 1. End-to-end architecture and training pipeline of the SGLD Uncertainty-Aware GRU model.

The pipeline consists of seven sequential stages:

- Stage 1 – Input: Raw OHLCV time series of the target instrument.
- Stage 2 – Parallel preprocessing: (2A) construction of 13 multi-scale technical features and (2B) algorithmic generation of regime labels via extrema detection.
- Stage 3 – Sequence construction: formation of overlapping windows of length T = 10 and standardization.
- Stage 4 – Training phase: RegimeGRU network optimized first with Adam (burn-in) then with SGLD; posterior weight snapshots are retained.
- Stage 5 – Inference & uncertainty: Monte-Carlo averaging of softmax outputs yields predictive mean and percentile-based spreads.
- Stage 6 – Decision layer: high-confidence, low-uncertainty trough signals trigger long entries; peaks or elevated uncertainty trigger exits.
- Stage 7 – Output: executable trading signals and risk-managed positions.

3. Data Pipeline and Feature Engineering

3.1 Input Features

Thirteen engineered features are computed from OHLCV data using TA-Lib and rolling statistics. Features are partitioned into micro (short-horizon) and macro (longer-horizon) groups to capture multi-scale market dynamics.

Micro Features (7)

- Micro_RSI: 14-period Relative Strength Index normalized to [0, 1]
- Micro_RSI_Slope: 3-period difference of Micro_RSI (momentum of momentum)
- Micro_FastK / Micro_FastD: Stochastic Fast %K (period 5) and %D (period 3), normalized to [0, 1]
- Micro_Prc_Dev: Percentage deviation of Close from 20-period Simple Moving Average
- Micro_Vol_Burst: Current volume relative to 20-period volume SMA (volume expansion)
- Micro_Ret_Accel: One-day return minus its 5-period rolling mean (acceleration)

Macro Features (6)

- Macro_RSI: 50-period RSI normalized to [0, 1]
- Macro_FastK / Macro_FastD: Longer-horizon Stochastic (FastK period 20, FastD period 5)
- Macro_Prc_Dev: Percentage deviation of Close from 200-period SMA
- Macro_Vol_Base: Ratio of 20-period volume SMA to 100-period volume SMA
- Macro_Ret_Vel: 10-day price velocity (Close_t / Close_{t-10} − 1)

3.2 Sequence Construction

Features are standardized using a StandardScaler fitted exclusively on the training partition. Overlapping sequences of length T = 10 are formed. The label associated with each sequence is the regime label of the final timestep in the window. This produces input tensors of shape (N, 10, 13).

4. Ground-Truth Labeling of Extrema

Labels are generated algorithmically from price series without look-ahead bias beyond a short local refinement window.

- A centered 7-period SMA of Close is computed to smooth noise.
- Coarse peaks and troughs are identified via scipy.signal.find_peaks with minimum distance 7 and prominence equal to 3 % of the mean price level.
- Each coarse extremum is refined to the true local maximum or minimum within a ±1 bar window of the original price series.
- Dynamic padding is applied: each refined extremum index receives a label pad of at most 1 bar on each side, subject to non-overlap constraints with neighboring extrema.
- All remaining bars receive the neutral / trend label (class 2).

The resulting three-class regime series constitutes the supervised target for the sequential model.

5. Neural Architecture: RegimeGRU

The predictive model is a compact single-layer GRU preceded by a linear embedding (bottleneck) layer.

5.1 Layer Specification

Let x ∈ ℝ^{B × T × F} denote a batch of sequences (F = 13, T = 10). The forward computation proceeds as follows:

$$h\_emb = ReLU(x\ W\_emb + b\_emb\ ) \in \mathbb{R}\hat{}\{B \times T \times D\_b\}$$

where D_b = max(4, ⌊F/2⌋ + 2) = 8 is the bottleneck dimension.

$$h\_t\ ,\ h\_final = GRU(h\_emb)\ ,\quad h\_final \in \mathbb{R}\hat{}\{B \times H\}$$

with hidden dimension H = 32 and a single GRU layer (batch_first = True). Only the final hidden state is retained.

$$logits = h\_final\ W\_fc + b\_fc \in \mathbb{R}\hat{}\{B \times 3\}$$

The three logits correspond to the unnormalized scores for classes {Trough, Peak, Trend}.

5.2 Implementation Skeleton

```
class RegimeGRU(nn.Module):
    def __init__(self, input_features=13, hidden_dim=32, output_classes=3):
        self.bottleneck_dim = max(4, (input_features // 2) + 2)
        self.embedding = nn.Linear(input_features, self.bottleneck_dim)
        self.gru = nn.GRU(self.bottleneck_dim, hidden_dim, num_layers=1, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_classes)

    def forward(self, x):
        emb = torch.relu(self.embedding(x))
        out, _ = self.gru(emb)
        logits = self.fc(out[:, -1, :])
        return logits, out[:, -1, :]
```

6. Stochastic Gradient Langevin Dynamics (SGLD)

6.1 Theoretical Background

SGLD is a Markov Chain Monte Carlo method that injects carefully scaled Gaussian noise into the stochastic gradient updates, thereby converting ordinary SGD into an approximate sampler from the Bayesian posterior p(θ | D) ∝ p(D | θ) p(θ).

Under a Gaussian prior (equivalently L2 weight decay) the continuous-time limit of the discrete dynamics converges to the Langevin diffusion whose invariant measure is the target posterior.

6.2 Discrete Update Rule

The parameter update implemented by the custom SGLDOptimizer is:

$$g\_t \leftarrow \nabla\_\theta\ L(\theta\_t;\ B\_t) + \lambda\ \theta\_t$$

$$\theta\_\{t+1\} \leftarrow \theta\_t - \varepsilon\ g\_t + \eta\_t\ ,\quad \eta\_t \sim \mathcal{N}(0,\ 2\ \varepsilon\ \sigma^2\ I)$$

where L is the weighted cross-entropy loss on mini-batch B_t, λ is the weight-decay coefficient, ε is the learning rate, and σ is a noise-scale hyper-parameter that modulates effective temperature.

6.3 Training Schedule

- Epochs 0–149 (burn-in): standard Adam optimizer (lr = 1e-3, weight_decay = 1e-4) for rapid movement toward a high-probability region.
- Epochs 150–299: switch to SGLDOptimizer (lr = 5e-4, noise_scale = 0.001).
- After burn-in, model state dictionaries are recorded every thinning = 5 epochs, yielding approximately 30 posterior samples.
- Class-balanced cross-entropy loss is employed; inverse-frequency weights are computed from the training label histogram.

6.4 Optimizer Implementation

```
class SGLDOptimizer(torch.optim.Optimizer):
    def step(self, closure=None):
        for group in self.param_groups:
            for p in group['params']:
                if p.grad is None: continue
                if group['weight_decay'] != 0:
                    p.grad.data.add_(p.data, alpha=group['weight_decay'])
                lr = group['lr']
                noise = torch.randn_like(p.data) * math.sqrt(2.0 * lr) * group['noise_scale']
                p.data.add_(p.grad.data, alpha=-lr)
                p.data.add_(noise)
```

7. Uncertainty Quantification

For a new input sequence x, each collected posterior sample θ^{(s)} produces a softmax probability vector π^{(s)}(x) = softmax(f_θ^{(s)}(x)).

$$\mu(x) = (1/S) \sum_{s=1}^{S} \pi^{(s)}(x) \qquad \text{(mean predictive probability)}$$

$$\text{spread}_c(x) = Q_{0.95}(\pi_c) - Q_{0.05}(\pi_c) \qquad \text{(per-class 90 \% predictive interval width)}$$

The scalar uncertainty metric used downstream is the maximum spread across the three classes: SGLD_Spread(x) = max_c spread_c(x). In addition, a decision margin is defined as

$$\text{Margin}(x) = \max_c \mu_c(x) - \text{second\_max}_c \, \mu_c(x)$$

High margin indicates a confident class ranking; low spread indicates that the ranking is stable across posterior samples.

8. Decision Logic and Risk Filters

Trading signals are generated from the previous bar's predictive statistics (no same-bar look-ahead).

- Trough (long entry) condition: μ_Trough is the largest component of μ, Margin exceeds a calibrated threshold τ_m, and SGLD_Spread is below a buy-confidence threshold τ_b.
- Peak (exit) condition: μ_Peak is the largest component and Margin > τ_m, or the uncertainty veto SGLD_Spread > τ_v is triggered.

The triple (τ_m, τ_b, τ_v) is selected by exhaustive grid search on the validation window, maximizing a risk-adjusted performance metric. Optimal parameters are frozen and applied unchanged to the out-of-sample test window.

9. Summary of Key Hyper-parameters

Component	Value	Rationale
Sequence length T	10	Short-term temporal context
Input features F	13	7 micro + 6 macro indicators
Bottleneck dimension	8	Dimensionality reduction
GRU hidden size H	32	Capacity vs. regularization
Output classes	3	Trough / Peak / Trend
Burn-in epochs	150	Adam exploration phase
Total epochs	300	SGLD sampling after burn-in
Thinning interval	5	Decorrelated posterior samples
SGLD learning rate ε	5×10^{-4}	Step size for Langevin dynamics
Noise scale σ	0.001	Temperature control
Weight decay λ	1×10^{-4}	Gaussian prior strength
Mini-batch size	96	Stochastic gradient noise

10. Concluding Remarks

The presented architecture unifies multi-scale technical feature construction, sequential modeling via a compact GRU, and Bayesian uncertainty quantification through SGLD posterior sampling. The explicit separation of predictive mean and predictive spread permits a principled risk-managed trading policy that enters only on high-confidence, low-uncertainty trough signals and exits either on confident peak signals or elevated epistemic uncertainty.

All design choices—feature set, sequence length, network capacity, SGLD schedule, and decision thresholds—are fully specified and reproducible from the accompanying source notebook.